Technique	Description
Weight initialization	Xavier, He initialization
Regularization	L1/L2 regularization
Dropout	Randomly drop neurons during training
Batch normalization	Normalize layer outputs
Hyperparameter tuning	Change hidden layer sizes, number of layers

Goal: Push accuracy from ~83% to >90%

---

8. Important Notes

1. **Data type matters:** Features → `float32`, Labels → `long`
2. **No explicit Softmax** when using `CrossEntropyLoss`
3. **Always call `optimizer.zero_grad()`** before `loss.backward()` (gradients accumulate)
4. **`model.eval()`** and **`torch.no_grad()`** for evaluation
5. **Reproducibility:** Set random seeds for consistent results

---

End of Notes. Happy Learning!

PyTorch GPU Training Notes – Fashion MNIST Example

Topic 1: Why Use GPU for Deep Learning?

Explanation (In-depth)

Deep learning models often work with large datasets containing thousands or even millions of images. Training such models on a CPU is very slow because CPUs are designed for general-purpose tasks, not massive parallel computations. GPUs (Graphics Processing Units), on the other hand, have thousands of small cores that can perform many calculations simultaneously. This makes them ideal for the matrix multiplications and tensor operations that form the backbone of neural networks.

In the Fashion MNIST example:

- Total images: 70,000 (60,000 training + 10,000 test)
- When using a CPU, training on the full dataset is too slow, so we often take only a subset (e.g., 10,000 images). This gives lower accuracy (~83%).

- With a GPU, we can use the **entire** 60,000 training images, leading to better accuracy (~89% as shown in the video).

Code (to enable GPU in Google Colab)

```
# Step 1: Change runtime type in Colab
# Go to Runtime → Change runtime type → Select "GPU"
```

Output (screenshot description)

No code output, but after changing the runtime, you will see “GPU” enabled in the notebook info.

Topic 2: Check GPU Availability and Set Device

Explanation (In-depth)

Before moving any tensors or models to the GPU, we must check if a GPU is actually available. PyTorch provides the `torch.cuda.is_available()` function. If it returns `True`, we set the device to `"cuda"`; otherwise, we fall back to `"cpu"`. This makes the code portable – it will run on any machine, using GPU if present, else CPU.

We store the device in a variable (commonly called `device`) so that we can later use it to move models and data.

Code

```
import torch

# Check if GPU is available, otherwise use CPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

Output

```
Using device: cuda
```

(If GPU is not available: `Using device: cpu`)

Topic 3: Move the Model to GPU

Explanation (In-depth)

A PyTorch model consists of layers with weights and biases. These parameters are stored as tensors. By default, they are created on the CPU. To perform forward and backward passes on

the GPU, we must move the entire model to the GPU memory. This is done with the `.to(device)` method.

Once the model is on the GPU, all subsequent operations (like `model(images)`) will automatically happen on the GPU, as long as the input data is also on the GPU.

Code

```
# Assume we have a model defined (e.g., a simple neural network)
class FashionMNISTModel(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = torch.nn.Flatten()
        self.linear_relu_stack = torch.nn.Sequential(
            torch.nn.Linear(28*28, 128),
            torch.nn.ReLU(),
            torch.nn.Linear(128, 10)
        )
    def forward(self, x):
        x = self.flatten(x)
        logits = self.linear_relu_stack(x)
        return logits

model = FashionMNISTModel()

# Move the model to the GPU (or CPU if GPU not available)
model = model.to(device)
print(f"Model is on: {next(model.parameters()).device}")
```

Output

```
Model is on: cuda:0
```

Topic 4: Move Batches of Data to GPU Inside Training Loop

Explanation (In-depth)

The training loop iterates over batches provided by a DataLoader. Each batch contains features (images) and labels. By default, these are loaded as CPU tensors. To perform the forward pass, loss computation, and backpropagation on the GPU, we must move each batch to the same device as the model.

The typical line inside the inner loop is:

```
batch_features = batch_features.to(device)
batch_labels = batch_labels.to(device)
```

We reassign the moved tensors to the same variable names. This step is crucial – if you forget it, PyTorch will complain about mismatched devices.

Code (inside training loop)

```
# Assume we already have:
#   train_loader, model, loss_fn, optimizer

model.train()
for epoch in range(num_epochs):
    for batch_features, batch_labels in train_loader:
        # Move data to GPU (or CPU)
        batch_features = batch_features.to(device)
        batch_labels = batch_labels.to(device)

        # Forward pass
        predictions = model(batch_features)
        loss = loss_fn(predictions, batch_labels)

        # Backward pass & optimization
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

Output (example)

No direct output, but training speed increases significantly. You will see epochs completing much faster than on CPU.

Topic 5: Move Data to GPU in Evaluation Loop

Explanation (In-depth)

During evaluation (or testing), we also need to move the batches to the GPU. Even though we do not compute gradients, the forward pass must happen on the same device as the model. The code is identical to the training loop: inside the `with torch.no_grad():` block, we move features and labels to the device.

We also set the model to evaluation mode using `model.eval()` to disable dropout/batch norm specific behaviors.

Code

```
model.eval()
correct = 0
total = 0

with torch.no_grad():
```

```
    for batch_features, batch_labels in test_loader:
        # Move data to GPU
        batch_features = batch_features.to(device)
        batch_labels = batch_labels.to(device)

        outputs = model(batch_features)
        _, predicted = torch.max(outputs, 1)
        total += batch_labels.size(0)
        correct += (predicted == batch_labels).sum().item()

accuracy = 100 * correct / total
print(f"Test Accuracy: {accuracy:.2f}%")
```

Output (example)

```
Test Accuracy: 89.20%
```

(This is an improvement from the CPU-only accuracy of ~83% when using a subset of data.)

Topic 6: Additional Performance Tips

Explanation (In-depth)

Two extra settings can further speed up GPU training:

1. **Larger batch sizes** – GPUs have large memory bandwidth. Using bigger batches (e.g., 64, 128 instead of 32) allows better utilisation of the GPU's parallel cores. However, ensure the batch fits in GPU memory.
2. **Pin memory (`pin_memory=True` in DataLoader)** – When `pin_memory=True`, the DataLoader copies tensors into **pinned (page-locked) memory** on the CPU. Pinned memory can be transferred to the GPU much faster than ordinary pageable memory because the OS does not need to swap it. This reduces the overhead of moving each batch from CPU to GPU, especially when the dataset is too large to fit entirely in GPU memory.

Code

```
# Using a larger batch size and pin_memory
batch_size = 128

train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=batch_size,
    shuffle=True,
    pin_memory=True          # <-- important for faster GPU transfer
)
```

```
test_loader = torch.utils.data.DataLoader(
    test_dataset,
    batch_size=batch_size,
    shuffle=False,
    pin_memory=True
)
```

Output

No direct output, but training becomes noticeably faster, especially for large datasets.

Summary of Results

Condition	Accuracy	Remarks
CPU + subset of data	~83%	Used only ~10,000 images
GPU + full dataset	~89%	Used all 60,000 training images

The accuracy improved by about 6% simply by training on the full dataset using a GPU.

Complete Workflow Summary (Copy-ready checklist)

1. **Enable GPU** in Colab: `Runtime → Change runtime type → GPU`
2. **Set device**:
   `device = torch.device("cuda" if torch.cuda.is_available() else "cpu")`
3. **Move model**: `model = model.to(device)`
4. **Move data** inside training loop:
   `batch_features, batch_labels = batch_features.to(device), batch_labels.to(device)`
5. **Move data** inside evaluation loop (same as above)
6. **Optional speed-ups**:
   - Increase batch size (e.g., 128)
   - Set `pin_memory=True` in DataLoader

Final Note

Training on GPU is essential for deep learning with real-world datasets. The steps above are universal – you can apply them to any PyTorch project. Always remember to move **both the model and each batch of data** to the same device.